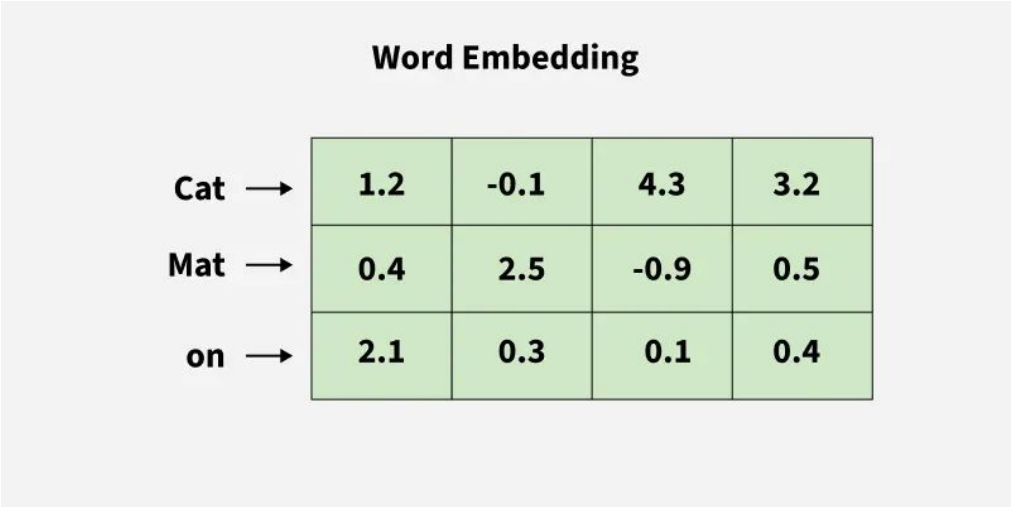

# Deep Learning

Lecture on RNNs
UG-3

# Machine Learning for text

1. Word embeddings: Text to numbers

2. Text processing: Classification/generation



# Word to vec: An approach for representing words and documents into numeric vectors.

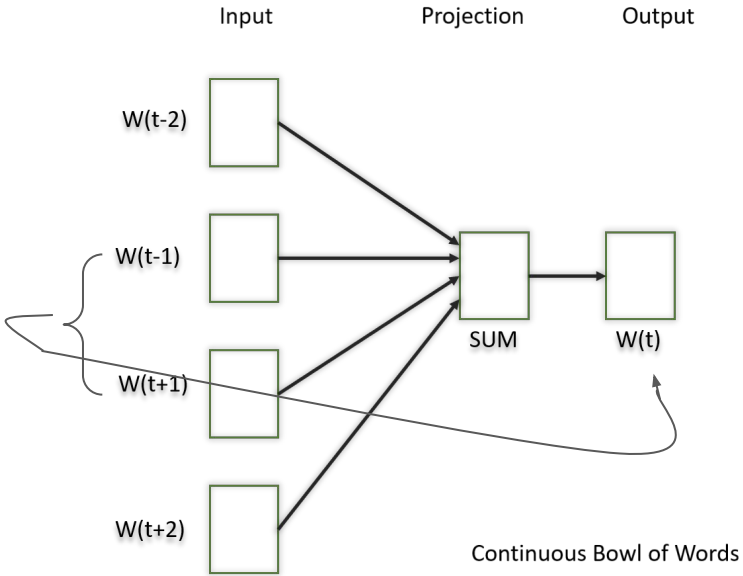
## One-hot encoding

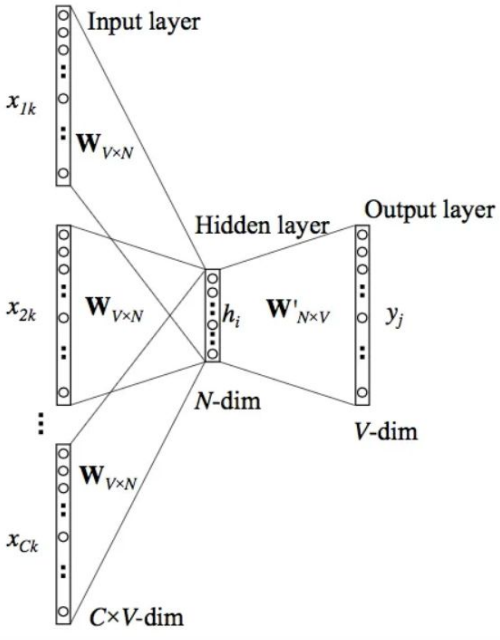
```
Vocabulary: {'mat', 'the', 'bird', 'hat', 'on', 'in', 'cat', 'tree', 'dog'}
Word to Index Mapping: {'mat': 0, 'the': 1, 'bird': 2, 'hat': 3, 'on': 4, 'in': 5, 'cat': 6,
'tree': 7, 'dog': 8}
One-Hot Encoded Matrix:
cat: [0, 0, 0, 0, 0, 0, 1, 0, 0]
in: [0, 0, 0, 0, 0, 1, 0, 0, 0]
the: [0, 1, 0, 0, 0, 0, 0, 0, 0]
hat: [0, 0, 0, 1, 0, 0, 0, 0, 0]
dog: [0, 0, 0, 0, 0, 0, 0, 0, 1]
on: [0, 0, 0, 0, 1, 0, 0, 0, 0]
the: [0, 1, 0, 0, 0, 0, 0, 0, 0]
mat: [1, 0, 0, 0, 0, 0, 0, 0, 0]
bird: [0, 0, 1, 0, 0, 0, 0, 0, 0]
in: [0, 0, 0, 0, 0, 1, 0, 0, 0]
the: [0, 1, 0, 0, 0, 0, 0, 0, 0]
tree: [0, 0, 0, 0, 0, 0, 0, 1, 0]
```

## Word2Vec

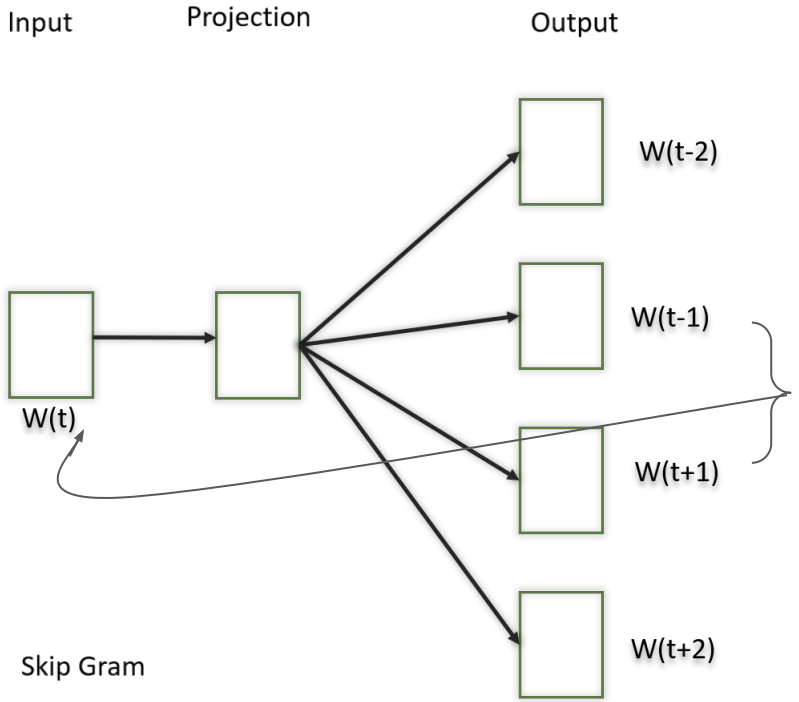
Word2Vec is a neural network-based model that generates dense vector representations of words.

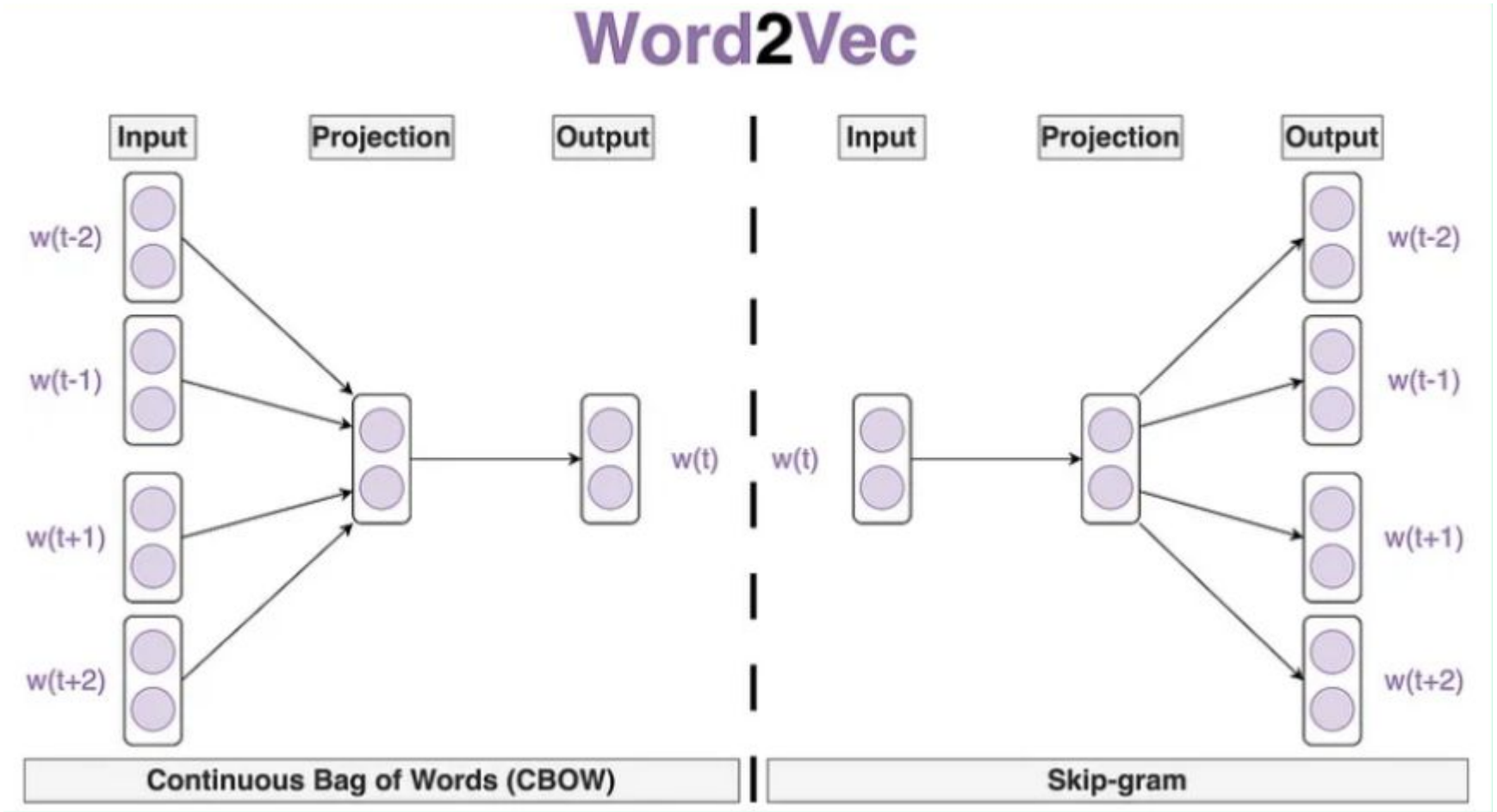
1. **Continuous Bag-of-Words (CBOW)**: Predicts the target word given its surrounding context words.



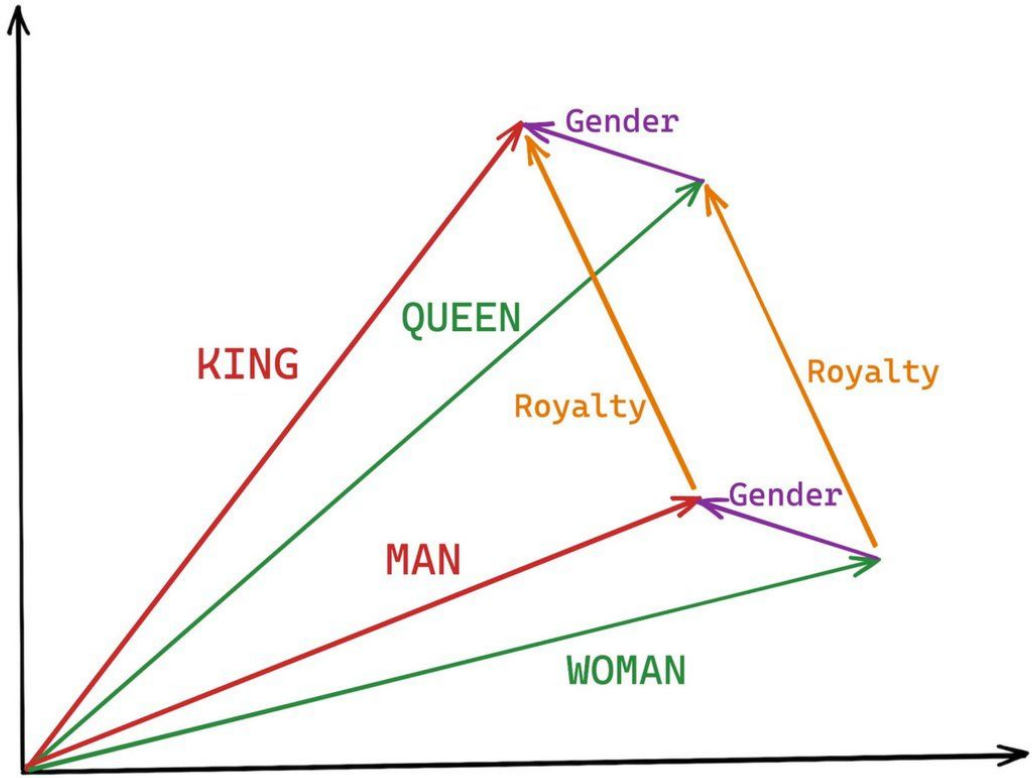



**2. Skip-Gram**: Predicts the surrounding context words given a target word. This is the opposite of the Continuous Bag of Words (CBOW) model.




Word2Vec
Input
Projection
Output
w(t-2)
w(t-1)
w(t+1)
w(t+2)
w(t)
Input
Projection
Output
w(t)
w(t-2)
w(t-1)
w(t+1)
w(t+2)
Continuous Bag of Words (CBOW)
Skip-gram

# Word Embedding space



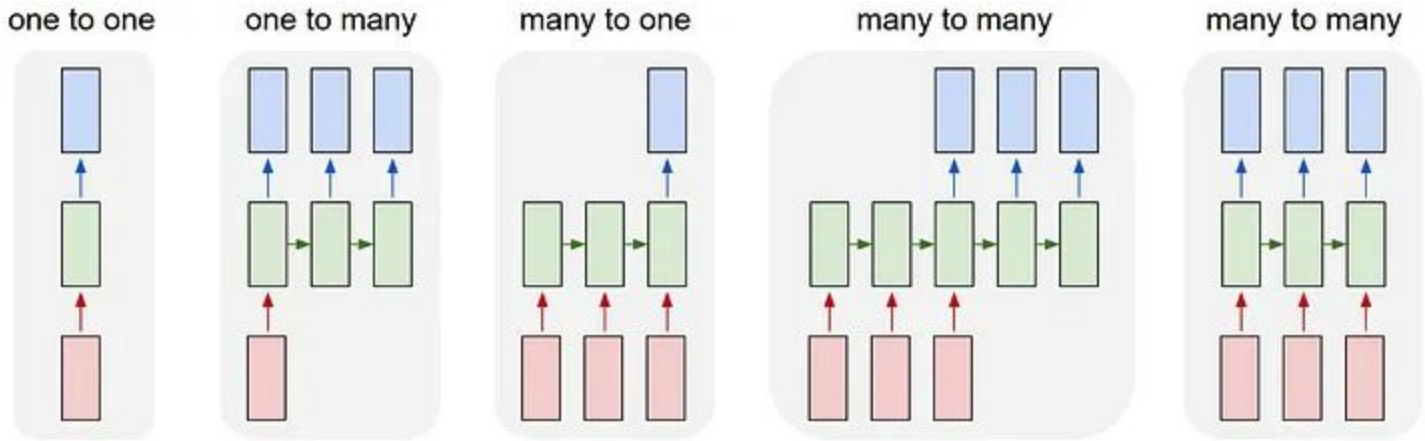
# Text Generation using RNNs and Transformers

**Text Generation** is the process where a model learns to produce natural language text: word by word or token by token

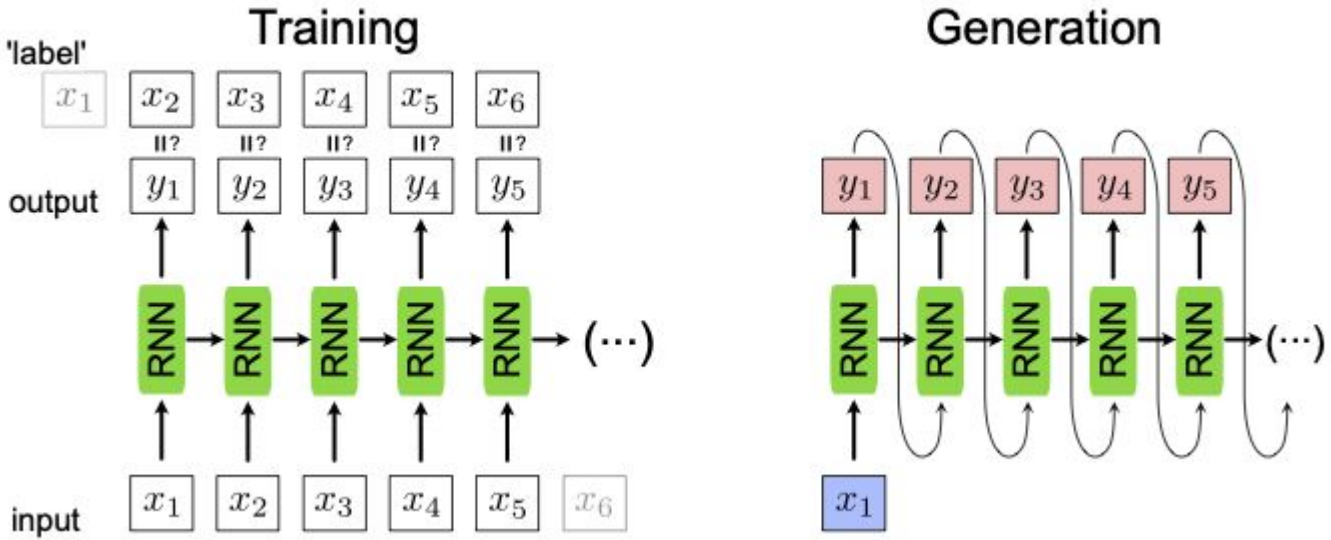
Formally, given a sequence of previous words $(w_1, w_2, \ldots, w_t)$, the model predicts the next word $w_{t+1}$:

$$P(w_{t+1}|w_1, w_2, ..., w_t)$$

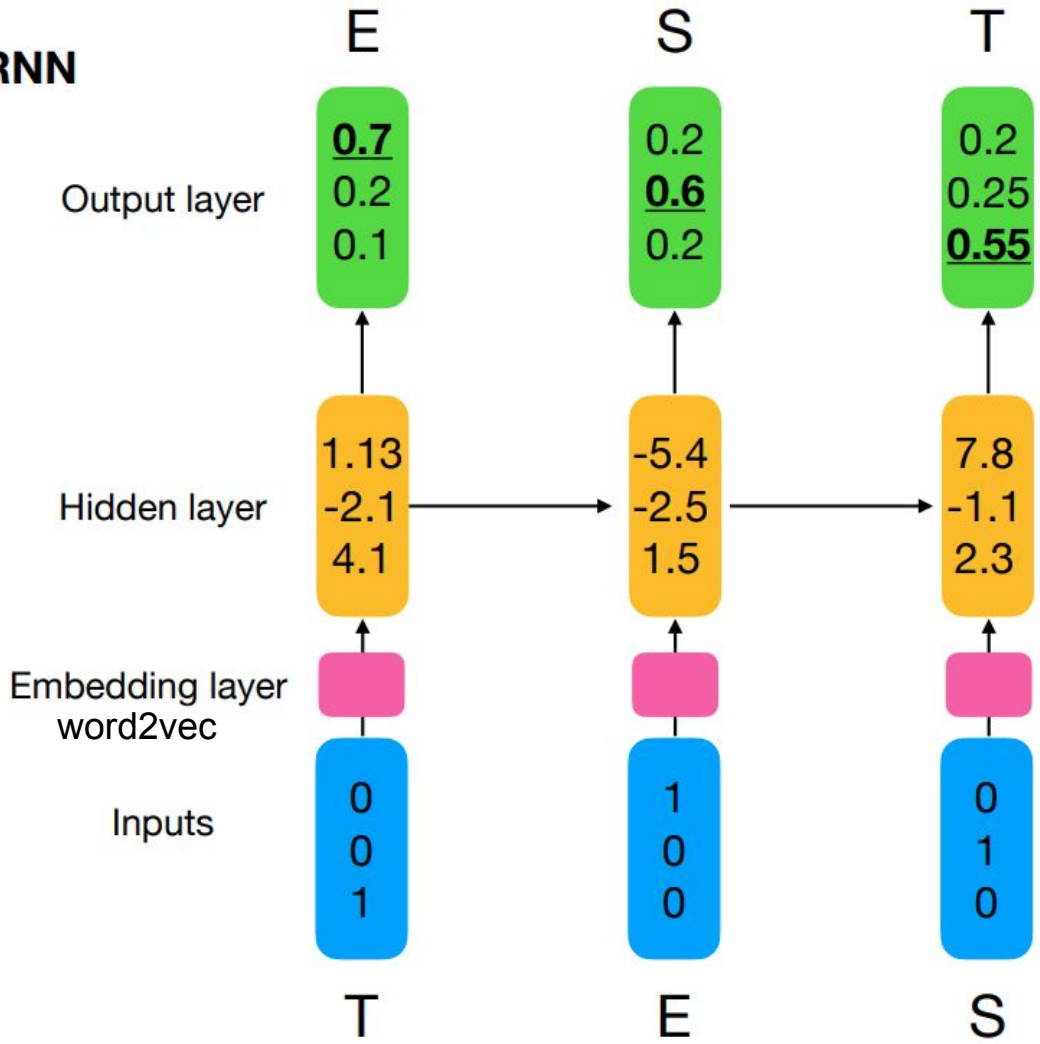
It repeats this recursively to generate a full sentence or paragraph.


one to one
one to many
many to one
many to many
many to many

# Text generation using RNNs



## Character RNN



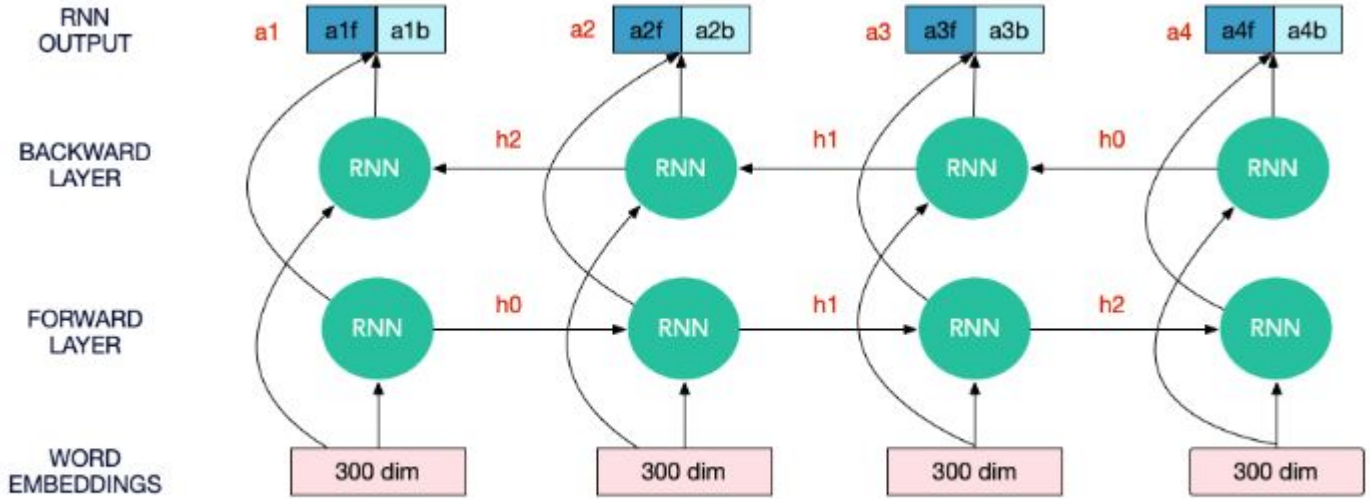
# Steps for training text generator

- Data preparation: Let the text is "*hello india*", prepare training set (input and targets) and let input sequence length is 4.

| X | Y |
|---|---|
| [h, e, l, l] | [o] |
| [e, l, l, o] | [ ] |
| [l, l, o, ] | [i] |
| [l, o, , i] | [n] |
| ... | ... |

- Choose a RNN architecture: hidden units, and
- initialize three matrices `W_hh, W_xh, W_hy`

| Item | Feed-Forward NN (BP) | Recurrent NN (BPTT) |
|---|---|---|
| **Forward pass** | $h = f(W^{(1)}x)$<br>$y = g(W^{(2)}h)$ | $a_t = W_{xh}x_t + W_{hh}h_{t-1}$<br>$h_t = f(a_t)$<br>$y_t = W_{hy}h_t$ |
| **Loss** | $L = \ell(y, \hat{y})$ | $L = \sum_{t=1}^{T} \ell_t(y_t, \hat{y}_t)$ |
| **Output-layer error** | $\delta^{(2)} = \dfrac{\partial L}{\partial a^{(2)}}$ | $\delta_t^{y} = y_t - \hat{y}_t$ |
| **Hidden-layer error** | $\delta^{(1)} = (W^{(2)})^{\top}\delta^{(2)} \odot f'(a^{(1)})$ | $\delta_t^{h} = \left[W_{hy}^{\top}\delta_t^{y} + W_{hh}^{\top}\delta_{t+1}^{h}\right] \odot f'(a_t),\ \delta_{T+1}^{h} = 0$ |
| **Weight update – output** | $\Delta W^{(2)} = -\eta\,\delta^{(2)}h^{\top}$ | $\Delta W_{hy} = -\eta \sum_{t=1}^{T} \delta_t^{y} h_t^{\top}$ |
| **Weight update – input/hidden** | $\Delta W^{(1)} = -\eta\,\delta^{(1)}x^{\top}$ | $\Delta W_{xh} = -\eta \sum_{t=1}^{T} \delta_t^{h} x_t^{\top}$ |
| **Weight update – recurrent** | — (no recurrence) | $\Delta W_{hh} = -\eta \sum_{t=1}^{T} \delta_t^{h} h_{t-1}^{\top}$ |


RNN OUTPUT
BACKWARD LAYER
FORWARD LAYER
WORD EMBEDDINGS
a1
a1f
a1b
a2
a2f
a2b
a3
a3f
a3b
a4
a4f
a4b
RNN
h2
h1
h0
h0
h1
h2
300 dim

## Bidirectional RNN

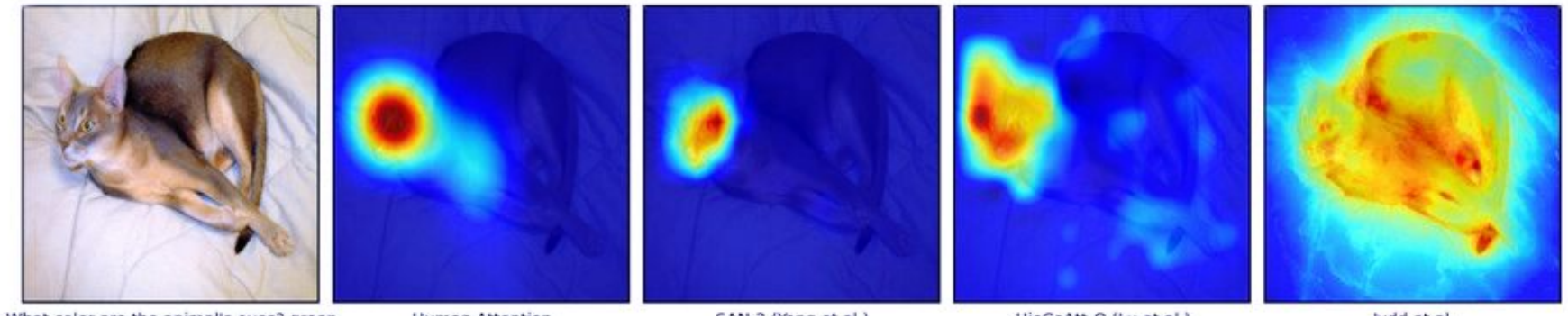
- Two separate hidden sequences:
  - **Forward**: $\overrightarrow{h_t}$ processes from $1 \to T$.
  - **Backward**: $\overleftarrow{h_t}$ processes from $T \to 1$.
- Output at each time $t$ uses **both**:

$$y_t = g\left(W_y[\overrightarrow{h_t}; \overleftarrow{h_t}]\right)$$

  where $[\ ;\ ]$ means concatenation.
- Parameters are usually:
  - Forward weights: $W_{xh}^{\rightarrow}, W_{hh}^{\rightarrow}$
  - Backward weights: $W_{xh}^{\leftarrow}, W_{hh}^{\leftarrow}$
  - Output weights: $W_{hy}$

Separate sets of weights for forward & backward

# Long sequence handling using attention



# A RNN Language Model

output distribution

$$\hat{y} = \text{softmax}(W_2 h^{(t)} + b_2)$$

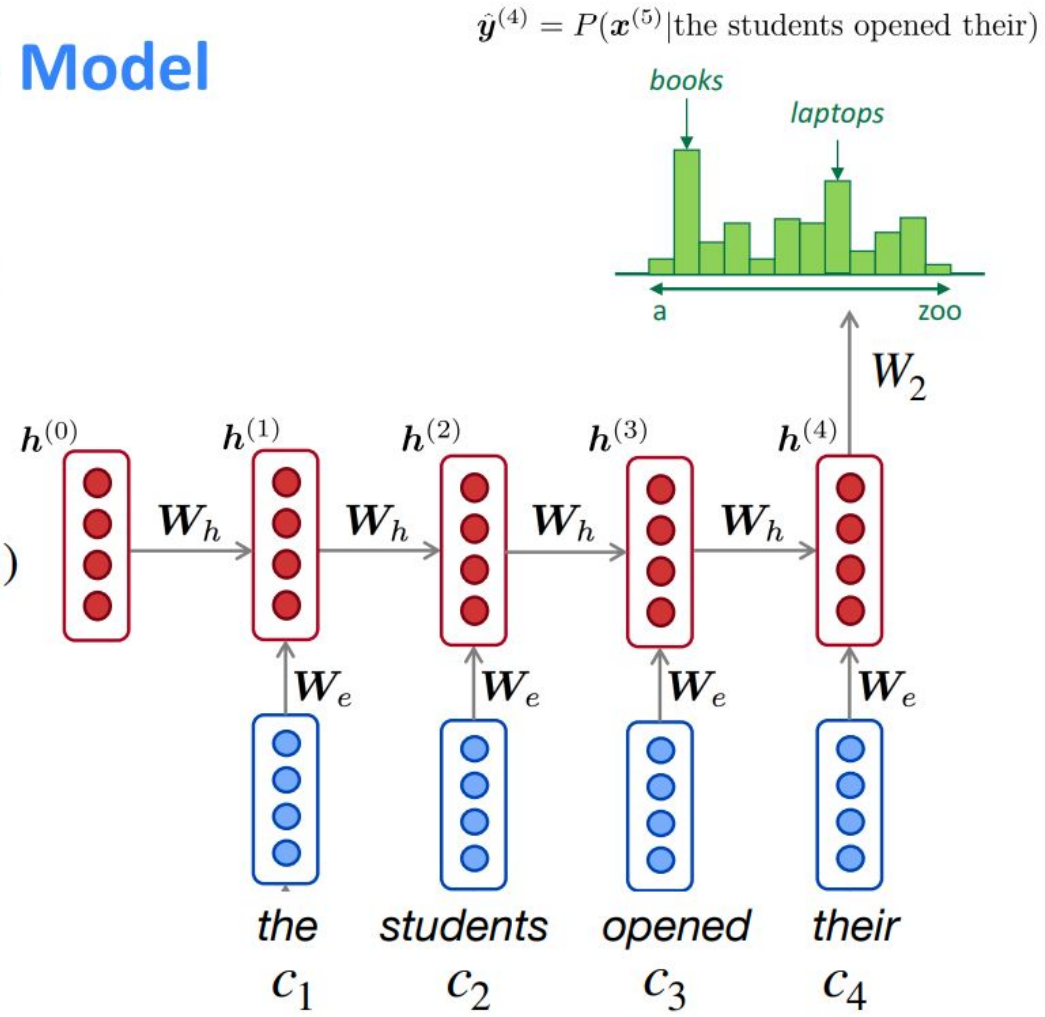
hidden states

$$h^{(t)} = f(W_h h^{(t-1)} + W_e c_t + b_1)$$

$h^{(0)}$ is initial hidden state!

word embeddings

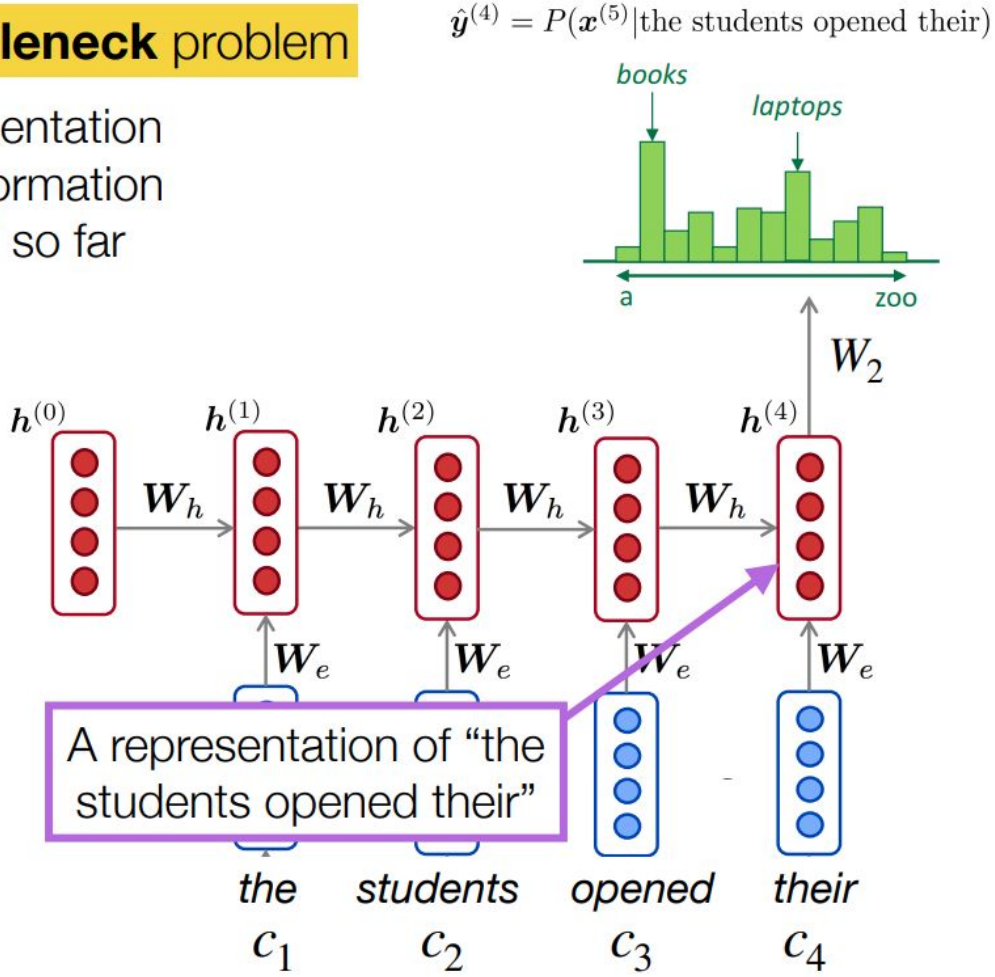
$$c_1, c_2, c_3, c_4$$



## RNNs suffer from a **bottleneck** problem

The current hidden representation must encode all of the information about the text observed so far

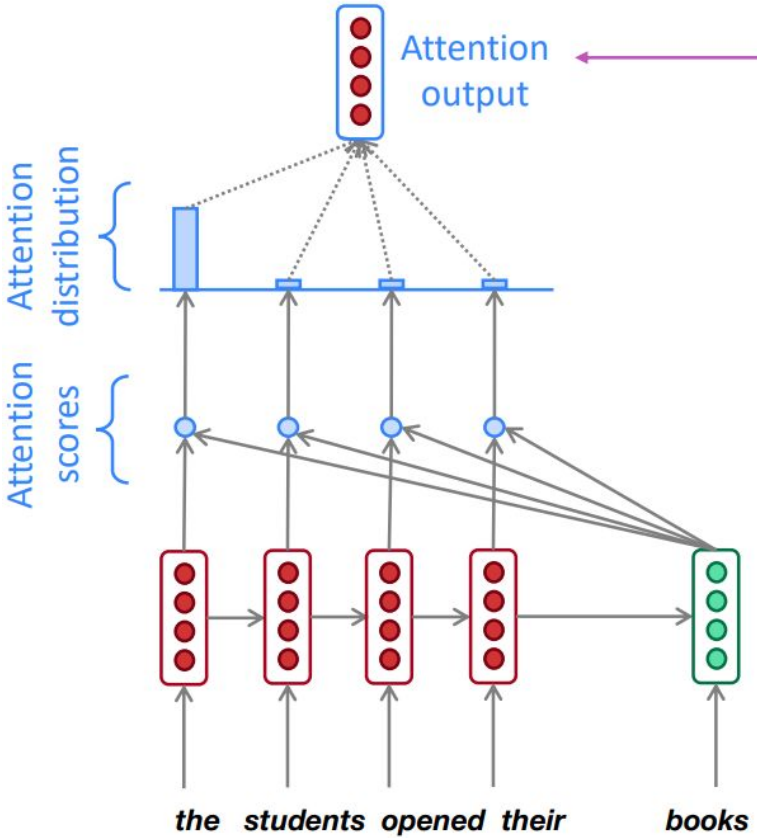
This becomes difficult especially with longer sequences



# The solution: **attention**

- **Attention mechanisms** (Bahdanau et al., 2015) allow language models to focus on a particular part of the observed context at each time step
  - Originally developed for machine translation, and intuitively similar to *word alignments* between different languages

# Attention mechanisms in neural language models




We use the attention distribution to compute a weighted average of the hidden states.

Intuitively, the resulting attention output contains information from hidden states that received high attention scores